

# Hash Tables

---

---

---

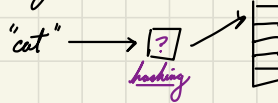
---

---

# Hash Tables

- Bucket Array

- Giant list of spots, "buckets," locations
- Each key is mapped to a single bucket using some sort of deterministic rule.  
(predictable)



- Hashing

- A hash function takes any kind of key & deterministically converts it into a number.

- Length of the key

| key       | idx $\Rightarrow$ $\ln(\text{key})$ |              |
|-----------|-------------------------------------|--------------|
| "cat"     | 3                                   | $\leftarrow$ |
| "guten"   | 6                                   |              |
| "ten"     | 3                                   | $\leftarrow$ |
| range(10) | 10                                  |              |
| (1, 2, 3) | 3                                   | $\leftarrow$ |

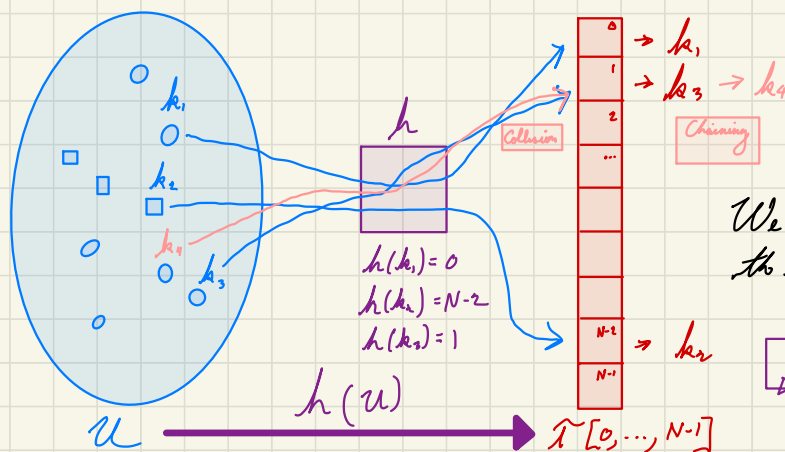
- Hash table:

1. Uses a hash function
2. Handles collision

| Data Structure        | Find                             | Insert                           | Delete                           | Space                                   |
|-----------------------|----------------------------------|----------------------------------|----------------------------------|-----------------------------------------|
| UnsortedArrayMap      | $\Theta(n)$                      | $\Theta(n)$                      | $\Theta(n)$                      | $\Theta(n)$                             |
| UnsortedLinkedListMap | $\Theta(n)$                      | $\Theta(n)$                      | $\Theta(n)$                      | $\Theta(n)$                             |
| SortedArrayMap        | $\Theta(\log(n))$                | $\Theta(n)$                      | $\Theta(n)$                      | $\Theta(n)$                             |
| SortedLinkedListMap   | $\Theta(n)$                      | $\Theta(n)$                      | $\Theta(n)$                      | $\Theta(n)$                             |
| BinarySearchTreeMap   | $\Theta(n)$<br>Also, $\Theta(h)$ | $\Theta(n)$<br>Also, $\Theta(h)$ | $\Theta(n)$<br>Also, $\Theta(h)$ | $\Theta(n)$                             |
| AVLTreeMap            | $\Theta(\log(n))$                | $\Theta(\log(n))$                | $\Theta(\log(n))$                | $\Theta(n)$                             |
| BucketArray           | $\Theta(1)$                      | $\Theta(1)$                      | $\Theta(1)$                      | Could be asymptotically larger than $n$ |
| Hash Table            | $\Theta(1)$                      | $\Theta(1)$                      | $\Theta(1)$                      | $\Theta(n)$                             |

Figure 2: Hello, constant runtime.

- **Universe ( $U$ )**: A set from which the keys will be taken from.
- **Hash Table ( $T[0, 1, \dots, N-1]$ )**: An array w/  $N$  slots.
- **Hash Function ( $h(u)$ )**: A function that maps keys from the universe into slots in the table.



We want a hash function that uniformly distributes keys.

→ A for that, when given a randomly chosen key, it will be equally likely to be mapped to any slot of  $T$ .

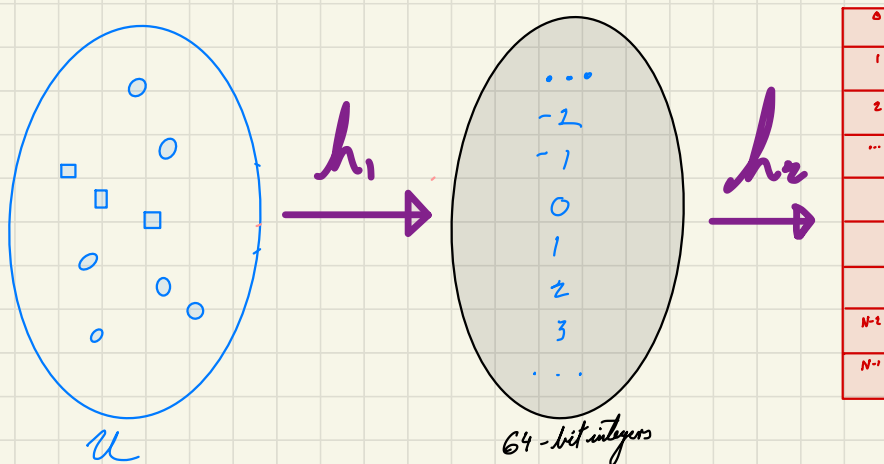
## Hash Functions

- A coding function ( $h_1$ ): A constant time process that transforms  $key \rightarrow integer$ .

$hash("cat") \# 9873452010298$ ; could even be negative

- A compression function ( $h_2$ ): Shrinks down output of  $h_1$  into a usable index.  $0 \leq index < capacity$ .

$$h(key) = h_2(h_1(key))$$



## Coding Functions

### 1. Integer casting

- Look at binary representation of key
- Take the 8 least significant bytes
- Interpret them as a 64-bit number

key = "cat"

'c'  $\rightarrow 99 \rightarrow 01100011$

'a'  $\rightarrow 97 \rightarrow 01100001$

't'  $\rightarrow 116 \rightarrow 01110100$

00000000 00000000 00000000 00000000 00000000 01100011 01100001 01110100

$0 \times 00000000000036174 \rightarrow 6501204$

"NYU 2025-0001"

"NYU 2025-0002"

"NYU 2025-0003"

...

'N' 'Y' 'U' '2' '0' '2' '5' '-'

Same hash value!

2. Component sum

- I. Take each byte (char) in key
- II. Cast to an integer
- III. Add them all together

$h_1("cat") \rightarrow 97 + 98 + 99 \rightarrow 294$

$h_i(\text{"act"}) \rightarrow$

$h_1$  ("two")

Not Uniform

### 3. Polynomial Accumulation

Let  $n$  be an integer  $\geq 2$ . For any key

$$h_1(\text{key}) = k_0 * z^0 + k_1 * z^1 + k_2 * z^2 + \dots + k_{m-1} * z^{m-1}$$

"C" → 99

"a" → 97

"f" → 116

7 → 33

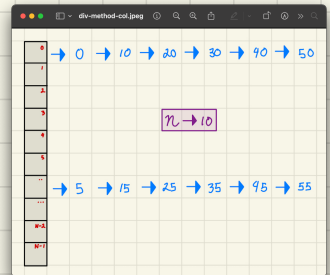
$$\text{hash} = 99 * 33^0 + 97 * 33^1 + 116 * 33^2$$
$$= 129624$$

## Compression Functions

- The division method:

$$h_2(k) = k \bmod N$$

$$= 129624 \% 50 = 24$$



$$\{0, 5, 10, 15, 20, 25, 30, 35, 40, 42, \cancel{50}, 55\} \quad N=10$$

# 1. The MAD Method

- Multiply, add, divide

- Let  $p$  be a prime number such that  $p > |U|$
- Let  $a$  be any integer from  $1 \rightarrow p-1$ .
- Let  $b$  be another integer from  $0 \rightarrow p-1$

$$h_z(k) = ((a * k + b) \bmod p) \bmod N$$

$$\{0, 5, 10, 15, 20, 25, 30, 35, 40, 42, 50, 55\} \quad N=10$$

$$p \rightarrow 101, \quad a = 31, \quad b = 6$$

$$h_z(k) = ((31k + 6) \bmod 101) \bmod 10$$

A screenshot of a digital notepad application with a grid background. It displays a table mapping keys (0-9) to values. The keys are listed in a column on the left, each in a small box with a red number. Blue arrows point from each key to its corresponding value. The values are: 5, 35, 50, 10, 25, 40, 55, 0, 15, 30, 45. At the bottom, a purple box contains the text 'N=10'.

|   |      |      |
|---|------|------|
| 0 | → 5  | → 20 |
| 1 | → 35 | → 50 |
| 2 |      |      |
| 3 | → 10 |      |
| 4 | → 25 | → 40 |
| 5 | → 55 |      |
| 6 | → 0  |      |
| 7 | → 15 | → 30 |
| 8 | → 45 |      |
| 9 |      |      |

$N=10$

## Runtime Analysis (for find())

1. Compute hash function
2. Look through chain in slot until we find (or not) key

$$O(n)$$

$$\text{Load Factor } \alpha = n/N \\ = 100/100 = 1$$

$$T(n) = O(1 + \alpha) \quad \alpha \leq 1 \\ \approx O(1+1) \quad \text{by resizing} \\ \approx O(2) \quad \text{the table} \\ \approx O(1)$$